Learning Rust Through a Portable Graph Loader, Edition 3

Architecture, Grust stores, backend adapters, functional Rust, and the By the Bay graph pipeline

---

Ownership, borrowing, data modeling, Grust stores, backend abstraction, and practical Rust design

Generated from the implementation in `src/main.rs`, `src/bin/load_graph.rs`, `src/graph_loader.rs`, and `scripts/enrich_meetup_entities.py`

June 2026

Contents

Preface

This third edition teaches Rust through one concrete implementation: the By the Bay graph pipeline. The project has two executables and one shared library module. One executable scrapes and extracts meetup and conference data. The other executable loads that neutral graph into several database backends: FalkorDB, HelixDB, LanceDB, pgGraph, Sail, and SurrealDB.

The important part is not the databases. The important part is the Rust design: how data moves through the program, how ownership is preserved, where borrowing matters, how errors are propagated, how the command line maps to runtime configuration, and how backend-specific behavior is hidden behind a stable API.

The first edition focused heavily on ownership and the borrow checker. Edition 2 broadened the view to architecture and backend adapters. Edition 3 keeps that material, but updates the story for the recent Grust abstraction: the local loader no longer hand-writes every backend protocol itself. It converts By the Bay data into Grust's portable graph model, selects a Grust graph store, and uses shared store traits to bootstrap, clear, and write the graph.

The code discussed here lives mainly in:

```
src/main.rs
src/bin/load_graph.rs
src/graph_loader.rs
```

1. The Shape of the Program

The loader has two main layers.

The binary, `src/bin/load_graph.rs`, owns command-line parsing and input conversion. It reads either:

- a consolidated export, `--input-format export`
- Claude-style per-talk records, `--input-format talk-records`

Both input paths produce the same neutral value from `grust::prelude`:

```
Graph {
    nodes: Vec<Node>,
    edges: Vec<Edge>,
}
```

The library module, `src/graph_loader.rs`, owns database loading. Its public API is intentionally small:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &Graph) -> Result<()>
```

That tells you the whole contract:

- The caller owns the configuration.
- The caller owns the graph data.
- The loader borrows both and delegates to the selected Grust store.
- The loader either succeeds or returns an error.

This is a good Rust boundary. It is explicit, cheap to call, and difficult to misuse.

2. Command-Line Parsing With `clap`

The binary starts with a struct that describes the CLI:

```
#[derive(Debug, Parser)]
#[command(version, about = "Load By the Bay graph JSON into a graph database")]
struct Args {
    #[arg(short, long, default_value = "data/bythebay-graph.json")]
    input: PathBuf,

    #[arg(long, value_enum, default_value_t = InputFormat::Export)]
    input_format: InputFormat,

    #[arg(long, value_enum, default_value_t = GraphBackend::Falkor)]
    backend: GraphBackend,

    #[arg(long, default_value = "data/lancedb")]
    lancedb_uri: String,

    #[arg(long, default_value = "host=127.0.0.1 user=postgres dbname=graph")]
    pggraph_connection_string: String,

    #[arg(long, default_value = "http://127.0.0.1:50051")]
    sail_endpoint: String,

    #[arg(long, default_value_t = 100)]
    batch_size: usize,
}
```

This is idiomatic Rust application code:

- `PathBuf` is an owned filesystem path.
- `usize` is the natural size type for counts and slice indexes.
- `InputFormat` and `GraphBackend` are enums, not strings.

Using enums matters. A stringly typed CLI lets invalid values drift deep into the program. A `ValueEnum` makes invalid values fail at parsing time.

```
#[derive(Debug, Clone, Copy, ValueEnum)]
enum InputFormat {
    Export,
    TalkRecords,
}
```

`Clone` and `Copy` are appropriate here because the enum is tiny and has no heap data. Passing it around by value is simpler than borrowing it.

3. Turning CLI Arguments Into Configuration

The CLI struct is specific to the binary. The library uses a separate config:

```
pub struct GraphLoadConfig {
    pub backend: GraphBackend,
    pub redis_url: String,
    pub helix_url: String,
    pub lancedb_uri: String,
    pub lancedb_table_prefix: String,
    pub pggraph_connection_string: String,
    pub pggraph_schema: String,
    pub pggraph_table_prefix: String,
    pub pggraph_auto_build: bool,
    pub pggraph_build_mode: PgGraphLoaderBuildMode,
    pub sail_endpoint: String,
    pub sail_user_id: String,
    pub sail_session_id: Option<String>,
    pub surreal_url: String,
    pub graph: String,
    pub replace: bool,
    pub batch_size: usize,
}
```

The conversion is implemented with `From`:

```
impl From<&Args> for GraphLoadConfig {
    fn from(args: &Args) -> Self {
        Self {
            backend: args.backend,
            redis_url: args.redis_url.clone(),
            helix_url: args.helix_url.clone(),
            lancedb_uri: args.lancedb_uri.clone(),
            lancedb_table_prefix: args.lancedb_table_prefix.clone(),
            pggraph_connection_string: args.pggraph_connection_string.clone(),
            pggraph_schema: args.pggraph_schema.clone(),
            pggraph_table_prefix: args.pggraph_table_prefix.clone(),
            pggraph_auto_build: args.pggraph_auto_build,
            pggraph_build_mode: args.pggraph_build_mode,
            sail_endpoint: args.sail_endpoint.clone(),
            sail_user_id: args.sail_user_id.clone(),
            sail_session_id: args.sail_session_id.clone(),
            surreal_url: args.surreal_url.clone(),
            graph: args.graph.clone(),
            replace: args.replace,
            batch_size: args.batch_size,
            // other fields omitted
        }
    }
}
```

This is one of the first places ownership matters.

`Args` owns its `String` fields. `GraphLoadConfig` also needs to own strings, because it may outlive the local `Args` borrow. Therefore, the conversion clones strings.

For small configuration strings, cloning is the right tradeoff. It avoids lifetime complexity and keeps the public config straightforward.

4. The Neutral Graph Model

The neutral graph model now comes from Grust:

```
pub struct Graph {
    pub nodes: Vec<Node>,
    pub edges: Vec<Edge>,
}

pub struct Node {
    pub label: String,
    pub id: String,
    pub props: Props,
}

pub struct Edge {
    pub label: Label,
    pub from: NodeId,
    pub to: NodeId,
    pub props: Props,
}
```

This is deliberately plain. A node has:

- one primary label
- a stable Grust node ID
- a duplicated `id` property for backends and query paths that use property lookup
- a property map

An edge has:

- relationship name
- source node ID
- target node ID
- optional edge properties

The database-specific record IDs are not part of this model. A Grust `Node` keeps the portable graph identity separate from whichever ID shape Falkor, Helix, LanceDB, pgGraph, Sail, or SurrealDB uses underneath.

Why `BTreeMap`?

Properties are stored in a `BTreeMap`:

```
pub type Props = BTreeMap<String, Value>;
```

A `HashMap` would also work. `BTreeMap` has one practical advantage here: deterministic iteration order. That makes generated files, table rows, backend requests, and test output more stable.

Determinism is useful in loader code because query generation is easier to debug when the same input produces the same string order.

5. Modeling Property Values

The loader does not leave arbitrary JSON unexamined at the boundary. It converts incoming `serde_json::Value` into Grust's portable property enum:

```
pub enum Value {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    String(String),
    StringArray(Vec<String>),
    Json(serde_json::Value),
}
```

This is a key design change from the earlier edition. The local loader used to support only `String` and `StringArray`. The Grust model is wider: booleans, numbers, nulls, strings, string arrays, and fallback JSON values can all cross the loader boundary.

That buys three things:

- each backend store handles a finite set of cases
- unsupported values are converted at the importer boundary
- tests can compare values directly with `PartialEq`

The implementation also defines convenient conversions:

```
impl From<String> for Value {
    fn from(value: String) -> Self {
        Self::String(value)
    }
}

impl From<&str> for Value {
    fn from(value: &str) -> Self {
        Self::String(value.to_string())
    }
}
```

This lets calling code write:

```
("name", person.name.clone().into())
```

instead of:

```
("name", Value::String(person.name.clone()))
```

6. Node IDs Without Borrowing Tricks

Every node has an owned Grust ID:

```
Node::new("Meetup", meetup.id.clone(), props)
```

The earlier loader kept the canonical ID inside `props["id"]` and had an `id(&self) -> Result<&str>` helper. Grust makes identity a first-class field on `Node`, so backends do not need to recover identity from the property map before writing.

The loader still duplicates the ID as a string property:

```
props.insert("id".to_string(), Value::from(node.id.clone()));
```

That duplication is intentional. The `Node.id` field is the portable graph identity. The `id` property is data visible to databases, indexes, and query examples. Rust makes the ownership cost explicit: when the same string must live in two places, the conversion clones it.

7. Error Handling With `anyhow`

The loader is an application-level tool, not a reusable low-level crate. It uses `anyhow::Result`:

```
use anyhow::{Context, Result, bail};
```

`bail!` returns an error immediately:

```
bail!("all graph nodes must include a string id property")
```

`Context` adds useful information to lower-level errors:

```
let graph_json = fs::read_to_string(input)
    .with_context(|| format!("failed to read {}", input.display()))?;
```

The `?` operator is central. It means:

- if the operation succeeds, unwrap the value
- if it fails, return the error from the current function

This keeps fallible code linear and readable.

8. Reading the Consolidated Export

The default importer reads one JSON file into typed structs:

```rust
fn read_export_graph(input: &PathBuf) -> Result<Graph> {
    let graph_json = fs::read_to_string(input)
        .with_context(|| format!("failed to read {}", input.display()))?;
    let export: GraphExport = serde_json::from_str(&graph_json)
        .with_context(|| format!("failed to parse {}", input.display()))?;
    export.to_graph_data()
}
```

Notice the ownership path:

1. `fs::read_to_string` returns an owned `String`.
2. `serde_json::from_str` borrows that string while parsing.
3. `GraphExport` owns the parsed data.
4. `to_graph_data` borrows `GraphExport` and creates owned `Graph`.

The export structs mirror the JSON:

```rust
#[derive(Debug, Deserialize)]
struct GraphExport {
    source_urls: Vec<String>,
    conferences: Vec<Conference>,
    meetups: Vec<MeetupGroup>,
    people: Vec<Person>,
    talks: Vec<Talk>,
    edges: Vec<Edge>,
}
```

These structs are private to the binary. That is a good boundary: the JSON format is an input concern, not a graph-loading concern.

9. Converting Export Records Into Nodes

Each export entity gets a small converter:

```
fn meetup_node(meetup: &MeetupGroup) -> Node {
    let mut props = props([
        ("id", meetup.id.clone().into()),
        ("name", meetup.name.clone().into()),
        ("url", meetup.url.clone().into()),
    ]);
    insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
    Node::new("Meetup", meetup.id.clone(), props)
}
```

This function borrows a `MeetupGroup` and returns an owned `Node`.

Why clone? Because the graph node must own its property strings. The source `MeetupGroup` is only borrowed. Moving fields out of a borrowed struct is not allowed, and should not be allowed: the caller still owns the export.

`as_deref` is a subtle useful method:

```
meetup.timezone.as_deref()
```

It turns `Option<String>` borrowed through `&MeetupGroup` into `Option<&str>`. That matches the helper:

```
fn insert_optional(props: &mut Props, key: &str, value: Option<&str>)
```

This avoids cloning optional strings unless they are actually present and non-empty.

10. Building Maps With Const Generics

The helper for property maps is:

```
fn props<const N: usize>(items: [(&str, Value); N]) -> Props {
    items
        .into_iter()
        .map(|(key, value)| (key.to_string(), value))
        .collect()
}
```

`const N: usize` lets the function accept arrays of any length while preserving static type information.

These all compile:

```
props([("id", "person:1".into())])

props([
    ("id", "meetup:1".into()),
    ("name", "Graph Night".into()),
    ("url", "https://example.test".into()),
])
```

Without const generics, you would likely accept a slice or vector. The array version is compact at call sites and allocation-free before the final map is collected.

11. Optional Properties and Mutable Borrows

Optional fields are inserted with:

```
fn insert_optional(props: &mut Props, key: &str, value: Option<&str>) {
    if let Some(value) = value.filter(|value| !value.is_empty()) {
        props.insert(key.to_string(), value.into());
    }
}
```

This function takes `&mut BTreeMap<...>`.

That mutable borrow means the caller temporarily gives the helper exclusive access to the map. While the helper is running, no other code can read or write the same map. After the function returns, the borrow ends and the caller can use the map again.

This is the borrow checker protecting you from iterator invalidation and concurrent mutation bugs.

12. Alternate Importer: Per-Talk Records

The alternate importer accepts Claude-style talk record files:

```
#[derive(Debug, Deserialize)]
struct TalkRecord {
    nodes: Vec<TalkRecordNode>,
    edges: Vec<TalkRecordEdge>,
}
```

It supports flexible field names:

```
#[derive(Debug, Deserialize)]
struct TalkRecordNode {
    id: String,
    #[serde(alias = "type", alias = "kind")]
    label: String,
    #[serde(default)]
    properties: serde_json::Map<String, serde_json::Value>,
}
```

`serde(alias = "...")` is useful when ingesting adjacent formats. It lets the Rust code keep one field name, `label`, while accepting JSON that says `type` or `kind`.

The importer deduplicates into maps:

```
let mut nodes_by_id: BTreeMap<String, Node> = BTreeMap::new();
let mut edges_by_key: BTreeMap<(String, String, String), Edge> = BTreeMap::new();
```

Node identity is the stable string ID. Edge identity is the tuple:

```
(from, to, relationship)
```

This mirrors how graph loaders usually think about idempotence.

13. Moving Values Into Deduplication Maps

The edge deduplication code is:

```
edges_by_key
    .entry((
        edge.from.clone(),
        edge.to.clone(),
        edge.relationship.clone(),
    ))
    .or_insert_with(|| talk_record_edge(edge));
```

This is a compact ownership lesson.

The map key needs owned strings, so the key clones the fields. If the edge is not already present, `or_insert_with` moves the original strings and optional edge properties into the stored `Edge`.

Why not avoid the clones? You could restructure this code to clone less, but this version is simple and correct. For a loader dominated by database I/O, the small string clones are not the bottleneck.

Rust makes the cost visible, which lets you choose intentionally.

14. Converting JSON Values Safely

The alternate importer converts arbitrary JSON into the loader's smaller property enum:

```
fn json_graph_value(value: serde_json::Value) -> Value {
    match value {
        serde_json::Value::Null => Value::Null,
        serde_json::Value::Bool(value) => Value::Bool(value),
        serde_json::Value::Number(value) => {
            if let Some(value) = value.as_i64() {
                Value::Int(value)
            } else if let Some(value) = value.as_f64() {
                Value::Float(value)
            } else {
                Value::Json(serde_json::Value::Number(value))
            }
        }
        serde_json::Value::String(value) => Value::String(value),
        serde_json::Value::Array(values) => {
            let strings = values
                .iter()
                .filter_map(|value| match value {
                    serde_json::Value::String(value) if !value.is_empty() => Some(value.clone()),
                    _ => None,
                })
                .collect::<Vec<_>>();
            if strings.len() == values.len() {
                Value::StringArray(strings)
            } else {
                Value::Json(serde_json::Value::Array(values))
            }
        }
        serde_json::Value::Object(value) => Value::Json(serde_json::Value::Object(value)),
    }
}
```

This is boundary normalization. Inside the loader, backends do not need to deal with arbitrary JSON first. They receive a typed Grust value and can decide how their storage layer represents each supported variant.

The function takes `serde_json::Value` by value, not by reference. That means it can move strings out of the JSON without cloning:

```
serde_json::Value::String(value) => Value::String(value)
```

If it took `&serde_json::Value`, this branch would need to clone the string.

15. Public API, Private Implementation

The public function stays small:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &Graph) -> Result<()> {
    let runtime = tokio::runtime::Runtime::new().context("failed to create Tokio runtime")?;
    runtime.block_on(load_graph_async(config, graph))
}
```

The internal dispatch is async because the selected Grust stores are async:

```
async fn load_graph_async(config: &GraphLoadConfig, graph: &Graph) -> Result<()> {
    match config.backend {
        GraphBackend::Falkor => load_falkor(config, graph).await,
        GraphBackend::HelixHttp => load_helix_http(config, graph).await,
        GraphBackend::HelixRustSdk => load_helix_sdk(config, graph).await,
        GraphBackend::Lancedb => load_lancedb(config, graph).await,
        GraphBackend::Pggraph => load_pggraph(config, graph).await,
        GraphBackend::Sail => load_sail(config, graph).await,
        GraphBackend::Surrealdb => load_surreal_http(config, graph).await,
        GraphBackend::SurrealdbRustSdk => load_surreal_sdk(config, graph).await,
    }
}
```

The public API is still synchronous, which keeps the binary simple. The async work is contained in the library wrapper.

The outside world does not need to know how backend dispatch works. It only needs `load_graph`. Keeping the async dispatch and store construction private gives the project freedom to change backend internals without breaking callers.

16. Why Not Return Trait Objects?

Another design would be:

```
fn backend_store(config: &GraphLoadConfig) -> Box<dyn GraphAdminStore>
```

That is useful when the selected backend must be stored and reused as a runtime object. This loader does not need that. It dispatches once, constructs the chosen store, performs the load, prints a report, and exits.

The current match is simpler:

- no heap allocation for a boxed trait object
- no dynamic dispatch beyond the simple call
- easy to read
- easy to debug

Rust rewards choosing the simplest abstraction that fits the lifetime of the problem.

17. Backend-Specific Ownership

Each backend function borrows the neutral graph and creates one store-specific config value. For LanceDB:

```
async fn load_lancedb(config: &GraphLoadConfig, graph: &Graph) -> Result<()> {
    let store = LanceDbGraphStore::connect(LanceDbConfig {
        uri: config.lancedb_uri.clone(),
        table_prefix: config.lancedb_table_prefix.clone(),
        batch_size: config.batch_size,
    })
    .await
    .map_err(grust_error)?;
    let report = load_with_admin_store(&store, config.replace, graph).await?;
    println!(
        "loaded {} nodes and {} edges into LanceDB at {} with table prefix '{}'",
        report.nodes, report.edges, config.lancedb_uri, config.lancedb_table_prefix
    );
    Ok(())
}
```

Important details:

- `config` and `graph` are borrowed.
- store config strings are cloned because the store owns its configuration.
- `connect` returns a backend-specific store.
- `map_err(grust_error)` adapts Grust's error type into `anyhow`.
- the graph itself is never copied before loading.

18. Shared Loading Through `GraphAdminStore`

The repeated loading lifecycle is now one generic helper:

```
async fn load_with_admin_store<S>(store: &S, replace: bool, graph: &Graph) -> Result<LoadReport>
where
    S: GraphAdminStore,
{
    store.bootstrap().await.map_err(grust_error)?;
    if replace {
        store.clear().await.map_err(grust_error)?;
    }
    store.put_graph(graph).await.map_err(grust_error)
}
```

This is the heart of the improved abstraction. The By the Bay code no longer needs separate bootstrap, clear, and write loops for every backend. Any store that implements `GraphAdminStore` gets the same lifecycle.

The trait bound is static dispatch. The compiler monomorphizes the helper for each concrete store type used by the match arms. No boxed trait object is needed because the store does not escape the backend function that created it.

19. Backend Differences Shape the Code

The same graph model is written differently by each database, but those differences now live mostly in Grust:

- FalkorDB stores a named graph through Redis/Falkor commands.
- HelixDB has HTTP and Rust SDK store paths.
- LanceDB stores durable local node and edge tables.
- pgGraph writes into PostgreSQL tables and can optionally build graph indexes.
- Sail writes through Spark Connect into Delta-style tables.
- SurrealDB has direct HTTP and Rust SDK store paths.

The By the Bay module still owns backend selection and user-facing flags, but backend protocol details have moved to the store adapters.

The abstraction works because all stores can honor the same portable contract:

- stable string node IDs
- labels or storage tables derived from node labels
- relationships derived from edge names
- property values represented with Grust `Value`

20. Backend Capabilities and Property Values

Earlier versions of the local loader normalized everything down to strings and string arrays because the hand-written backend writers had different property limits. Grust's `Value` enum now keeps more information:

```
Value::Null
Value::Bool(true)
Value::Int(42)
Value::Float(3.14)
Value::String("Graph Loading".to_string())
Value::StringArray(vec!["rust".to_string(), "graphs".to_string()])
Value::Json(serde_json::json!({"source": "raw"}))
```

Backends can still have different capabilities. A graph database may support arrays directly, while a table-oriented store may serialize some values. The important architectural change is that those decisions sit inside Grust's store adapters instead of being repeated in this project.

The By the Bay export still keeps `tags`, `tags_json`, and `tags_csv`. That is practical data design: richer backends can use `tags`, string-oriented systems can use `tags_csv`, and JSON-friendly systems can use `tags_json`.

21. Batching as Store Policy

Backends still expose batch size, but it is passed into store configuration:

```
LanceDbConfig {
    uri: config.lancedb_uri.clone(),
    table_prefix: config.lancedb_table_prefix.clone(),
    batch_size: config.batch_size,
}
```

The store implementation decides how to use that number. It may call `chunks(config.batch_size.max(1))` internally, build a database batch request, or buffer rows before flushing to a table. The public contract stays the same: `--batch-size` expresses the user's preferred write granularity.

22. Sync and Async Together

The public loader function is synchronous:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &Graph) -> Result<()>
```

All backend work now runs through a Tokio runtime:

```
let runtime = tokio::runtime::Runtime::new().context("failed to create Tokio runtime")?;
runtime.block_on(load_graph_async(config, graph))
```

This is pragmatic for a command-line loader. The binary can stay simple while stores use async where required.

For a long-running server, you would probably make the public API async instead of creating runtimes internally.

23. Lifetimes Without Writing Lifetimes

Most of this project does not write explicit lifetime parameters. Rust infers them.

For example:

```
fn insert_optional(props: &mut Props, key: &str, value: Option<&str>)
```

The references only need to live for the duration of the function call.

Another example is the graph loader itself:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &Graph) -> Result<()>
```

The references only need to live while the load is running. The function does not store them globally or return borrowed pieces of them.

You usually write explicit lifetimes only when a function returns a borrowed value and there is more than one possible input lifetime. This code mostly has obvious borrowing relationships, so elision keeps it readable.

24. Tests as Design Locks

The loader has tests for importer behavior that should not regress:

```
#[test]
fn converts_export_to_backend_neutral_graph() {
    let graph = export.to_graph_data().unwrap();
    assert_eq!(graph.nodes.len(), 4);
    assert_eq!(graph.edges.len(), 1);
}
```

This test is not just checking syntax. It captures the conversion contract from the scraper's consolidated export into a Grust `Graph`.

Another test locks the alternate importer behavior:

```
#[test]
fn converts_talk_records_to_backend_neutral_graph() {
    // Builds a TalkRecord directly and checks the resulting Graph.
}
```

That test verifies two important details: node IDs are preserved both as Grust IDs and as `id` / `nid` properties, and string arrays remain string arrays. Good tests describe intent. They make future refactoring safer.

25. Practical Borrow Checker Rules From This Project

Here are the borrow checker lessons this loader teaches.

Prefer borrowing large inputs:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &Graph) -> Result<()>
```

Return owned output when transforming formats:

```
fn read_export_graph(input: &PathBuf) -> Result<Graph>
```

Clone at boundaries when a new owner is needed:

```
("name", person.name.clone().into())
```

Take mutable references for helper functions that update a structure:

```
insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
```

Move values when consuming parsed input:

```
fn json_graph_value(value: serde_json::Value) -> Value
```

Pass batch policy into backend store configs:

```
batch_size: config.batch_size
```

26. How the Abstractions Were Chosen

The main abstraction is the neutral graph:

```
Graph
Node
Edge
Value
```

That abstraction exists because the scraper should not know how FalkorDB, HelixDB, LanceDB, pgGraph, Sail, or SurrealDB write graph data.

The second abstraction is backend selection:

```
GraphBackend
GraphLoadConfig
load_graph(...)
```

That abstraction exists because the CLI needs to select a backend at runtime.

The third abstraction is internal:

```
GraphAdminStore
```

That abstraction exists because backend implementations share a lifecycle: bootstrap storage, optionally clear old data, then write the graph.

A useful rule: abstractions should protect a real boundary. Here the real boundaries are:

- input format versus graph model
- Grust graph model versus backend storage
- public API versus private store construction

27. What Could Improve Next

The current implementation is intentionally practical. Good next steps would be:

- add automated live integration tests for all supported stores
- add backend-specific read helpers for common graph traversals
- add date/time-specific property conventions on top of Grust `Value`
- document pgGraph build modes with examples from real PostgreSQL runs
- add richer Sail examples that read back the generated Delta tables

Do not split files just to split files. Split when navigation, ownership, or testability improves.

Conclusion

This graph loader is a useful Rust learning project because it is small enough to understand and real enough to show the language's strengths.

It demonstrates:

- typed command-line parsing
- owned data models
- borrowed public APIs
- fallible conversion with `Result`
- structured backend dispatch
- shared store loading
- importer normalization
- tests that encode backend constraints

The most important Rust idea here is ownership at boundaries. Parse into owned data. Borrow while loading. Clone only when a second owner is actually needed. Move values when consuming an input format. Keep backend-specific details behind a stable interface.

That is the heart of writing clear Rust.

28. Edition 3: The Whole Project Architecture

The full project is more than the loader. It is a data pipeline with four architectural zones:

1. acquisition and extraction in `src/main.rs`
2. cached-download enrichment in `scripts/enrich_meetup_entities.py`
3. import-shape normalization in `src/bin/load_graph.rs`
4. backend-neutral graph loading in `src/graph_loader.rs`

The executable in `src/main.rs` owns the messy edge of the world. It knows about URLs, HTML, Meetup GraphQL, LLM extraction, hand-written parsers, raw source records, and the final consolidated export. The enrichment script owns the cached, offline pass that turns downloaded records into first-class speakers, talks, projects, companies, meetups, and graph-record nodes and edges. The loader binary owns the contract between exported data and the portable graph model. The library module owns the contract between the portable graph model and databases.

That split is the main architectural decision of the codebase. Unreliable input formats and backend-specific write protocols are not allowed to leak into each other. The scraper can improve its parsing rules without caring whether the graph is later written to FalkorDB, HelixDB, LanceDB, pgGraph, Sail, or SurrealDB. The database loader can add a backend without knowing how Meetup pages were scraped.

The project is arranged as a pipeline:

```
remote pages and APIs
    -> raw source records
    -> extracted structured records
    -> GraphExport JSON
    -> optional enriched entity records and graph-record export
    -> Grust Graph
    -> GraphAdminStore writes
```

Each stage consumes data that has become stable enough for that layer, then emits a simpler value for the next layer.

29. Crate Layout and Module Boundaries

The crate is named `bythebay-scraper`, but it exposes a reusable library module:

```
pub mod graph_loader;
```

That one-line `src/lib.rs` is small, but it changes the architecture. It lets the `load_graph` binary use the loader through the crate boundary:

```
use bythebay_scraper::graph_loader::{
    GraphBackend, GraphLoadConfig, PgGraphLoaderBuildMode, load_graph,
};
use grust::prelude::*;
```

This avoids copying loader code into the binary. It also makes public and private API explicit. `GraphBackend`, `GraphLoadConfig`, `PgGraphLoaderBuildMode`, and `load_graph` are public because they are the By the Bay loader contract. `Graph`, `Node`, `Edge`, `Value`, and the store traits come from Grust because graph portability is now shared outside this project.

The module boundary is deliberately thin:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &Graph) -> Result<()>
```

Everything behind it can change as long as this contract remains stable.

30. Executables and Scripts

The default binary in `src/main.rs` is the scraper/exporter. It uses Tokio and `reqwest::Client`, so its `main` function is asynchronous. Its job is broad: it fetches conference pages, fetches Meetup archives, chooses between LLM extraction and local parsing, writes split records under `data/raw`, builds `GraphExport`, and writes `data/bythebay-graph.json`.

The enrichment script is a separate offline executable:

```
python3 scripts/enrich_meetup_entities.py --batch-size 100
```

It reads cached source records from `data/raw/sources`, uses `data/bythebay-graph.json` as fallback context, processes Meetup downloads in batches, and writes aggregate, JSON Lines, per-entity, and graph-record outputs under `data/enriched`.

The loader binary in `src/bin/load_graph.rs` is narrower:

```
fn main() -> Result<()> {
    let args = Args::parse();
    let graph = read_graph_data(&args.input, args.input_format)?;
    load_graph(&GraphLoadConfig::from(&args), &graph)
}
```

That short `main` is a strong signal. The binary parses arguments, reads input, and delegates. The interesting loader behavior lives in the library, not in the executable wrapper.

31. Data Contracts: Raw, Export, Graph

The project uses several data shapes because one shape would be too vague:

- raw source records preserve what was fetched
- extracted records preserve what was understood from a source
- `GraphExport` is a stable interchange file
- enriched entity records preserve normalized speakers, talks, companies, projects, meetups, and conferences
- Grust `Graph` is the database-neutral loader model

In `src/main.rs`, structs such as `RawSpeaker`, `RawTalk`, `RawMeetup`, `RawMeetupEvent`, and `RawMeetupSession` reflect scraper concerns. They contain optional fields because real web data is incomplete.

In `src/bin/load_graph.rs`, `GraphExport` is closer to the graph:

```
struct GraphExport {
    source_urls: Vec<String>,
    conferences: Vec<Conference>,
    meetups: Vec<MeetupGroup>,
    people: Vec<Person>,
    talks: Vec<Talk>,
    edges: Vec<Edge>,
}
```

In `grust::prelude`, `Graph` removes domain-specific record names:

```
pub struct Graph {
    pub nodes: Vec<Node>,
    pub edges: Vec<Edge>,
}
```

Earlier stages are rich in domain vocabulary. Later stages are rich in graph vocabulary.

Enriched entity records

The enrichment output has two audiences. Humans and data tools can read the entity files:

```
data/enriched/bythebay-entities.json
data/enriched/speakers.json
data/enriched/talks.json
data/enriched/projects.json
data/enriched/companies.json
data/enriched/meetups.json
data/enriched/conferences.json
data/enriched/entities/{entity-type}/*.json
```

The loader can read the graph-record export with the existing `talk-records` importer:

```
cargo run --bin load_graph -- \
  --input data/enriched/bythebay-enriched-graph.json \
```

```
  --input-format talk-records \
  --backend falkor
```

That graph-record file uses `nodes` and `edges`, but its domain model is richer than the original `GraphExport`. It includes `Speaker`, `Company`, and `Project` nodes, and relationships such as `WORKS_FOR`, `MENTIONS_PROJECT`, `PRESENTED_BY_COMPANY`, `PART_OF_MEETUP`, and `PART_OF_CONFERENCE`.

The current cached run produced 1,218 talks, 309 directly affiliated speakers, 196 companies, 219 projects, 9 meetups, 2 conferences, 1,953 graph nodes, and 2,886 graph edges. Every speaker record has at least one directly extracted role or company affiliation, and 245 include a company affiliation. The script also writes batch summaries so a large cached corpus can be inspected incrementally.

32. `serde`: Deriving the Boundary

The codebase uses `serde` to turn external JSON into typed Rust values:

```
#[derive(Debug, Deserialize)]
struct TalkRecordNode {
    id: String,
    #[serde(alias = "type", alias = "kind")]
    label: String,
    #[serde(default)]
    properties: serde_json::Map<String, serde_json::Value>,
}
```

`derive` asks the compiler to generate trait implementations. `serde(alias = "...")` accepts alternate input field names. `serde(default)` makes missing fields explicit. The project uses dynamic `serde_json::Value` at the uncertain edge, then converts into typed structs when the shape is known.

33. Enums as Closed Worlds

The project uses enums where a value should be one of a known set:

```
pub enum GraphBackend {
    Falkor,
    HelixHttp,
    HelixRustSdk,
    Lancedb,
    Pggraph,
    Sail,
    Surrealdb,
    SurrealdbRustSdk,
}
```

This gives the compiler a complete list of backend choices. If a new backend is added, the match in `load_graph` points at every dispatch site that must be updated.

`Value` is another closed world:

```
pub enum Value {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    String(String),
    StringArray(Vec<String>),
    Json(serde_json::Value),
}
```

The loader intentionally converts unknown JSON into this finite set before calling a store. The enum is a portable property contract.

34. Traits as Backend Ports

The loader now relies on Grust's store trait:

```
async fn load_with_admin_store<S>(store: &S, replace: bool, graph: &Graph) -> Result<LoadReport>
where
    S: GraphAdminStore,
```

Each backend function creates a concrete store:

```
FalkorGraphStore
HelixHttpGraphStore
HelixSdkGraphStore
LanceDbGraphStore
PgGraphStore
SailGraphStore
SurrealHttpGraphStore
SurrealSdkGraphStore
```

This is a port-and-adapter design in compact Rust form. The trait names the port: "administer and write this graph." The structs are adapters: "write it into Falkor," "write it into LanceDB," "write it into Sail," and so on.

The trait itself is public in Grust because multiple projects can share it. The By the Bay loader still hides store construction behind its own small public API.

35. Store Backends and Transports

Some backends are the same database through different transports:

- `helix-http` uses Helix's dynamic-query HTTP path.
- `helix-rust-sdk` uses Helix's Rust SDK path.
- `surrealdb` uses direct SurrealQL HTTP.
- `surrealdb-rust-sdk` uses SurrealDB's Rust SDK path.

Other backends represent different storage engines:

- `falkor` stores a graph in FalkorDB.
- `lancedb` stores graph tables locally in LanceDB.
- `pggraph` stores graph data in PostgreSQL through pgGraph.
- `sail` writes through Spark Connect into Sail-managed tables.

The loader's enum names the user's choice. Grust owns the transport and protocol mechanics.

36. What Got Reused and Abstracted

The biggest reuse is now the store lifecycle:

```
store.bootstrap().await?;
if replace {
    store.clear().await?;
}
store.put_graph(graph).await?;
```

The By the Bay code reuses Grust's graph data model, store traits, backend configs, and load reports. It still owns CLI flags, source-format conversion, label lists, relationship lists, and the small mapping from `Args` to `GraphLoadConfig`.

This is a better abstraction than the previous edition's private `GraphLoader` trait because it is shared at the right level. Grust can improve one backend adapter without changing the By the Bay importer.

37. A Review of Grust

Grust is not just a dependency in this project. It is the portability layer that lets the By the Bay loader stop owning every backend dialect itself.

The top-level Grust facade is published as the `grust-graph` package on crates.io. Its library name is still `grust`, so application code keeps the plain `grust::prelude` import:

```
pub use grust_core::*;

#[cfg(feature = "falkor")]
pub use grust_falkor::*;

#[cfg(feature = "lancedb")]
pub use grust_lancedb::*;
```

The same pattern appears in `grust::prelude`. This is a good crate-boundary choice. The application imports one prelude, while Cargo features decide which backend crates are actually compiled. In `Cargo.toml`, the dependency uses a local crate alias for the published package:

```
grust = { package = "grust-graph", version = "0.1.0", features = [
  "cocoindex",
  "falkor",
  "helix",
  "lancedb",
  "pggraph",
  "sail",
  "surreal",
] }
```

The facade keeps application code small without forcing every backend dependency into every build, and the published package removes the old cross-repository path dependency from the meetup graph project.

Core Types

The strongest part of Grust is `grust-core`. It gives graph concepts their own types:

```
pub struct NodeId(String);
pub struct EdgeId(String);
pub struct Label(String);
```

These are newtypes over `String`. That is small, but valuable. A function that accepts a `NodeId` is not accidentally accepting a table name, URL, label, or free-form property key. The compiler gets more information about intent.

The portable graph is deliberately direct:

```
pub struct Graph {
    pub nodes: Vec<Node>,
    pub edges: Vec<Edge>,
}
```

`Node::new` also inserts the `id` property when it is missing:

```
props
    .entry("id".to_string())
    .or_insert_with(|| Value::from(id.as_str()));
```

That is a useful compatibility move. Backends get a first-class `NodeId`, while query languages and table formats that expect an `id` property still receive one.

The property value model is broader than the earlier local loader:

```
pub enum Value {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    String(String),
    StringArray(Vec<String>),
    Json(serde_json::Value),
}
```

This is a practical middle ground. It is not a full property-graph type system, but it preserves far more input information than a string-only model. It also gives backends a clear place to say "I can store this directly" or "I need to serialize this."

Builder and Deduplication

Grust includes a `GraphBuilder` with a default edge policy:

```
pub enum EdgePolicy {
    AllowDuplicates,
    DedupeByFromLabelTo,
}
```

The default dedupes edges by `(from, label, to)`. Nodes are stored in a `BTreeMap<NodeId, Node>`, and adding the same node ID with the same label extends the existing properties.

That choice matches this codebase's import problem: many raw records can mention the same talk, speaker, project, or company. The builder lets an importer merge repeated facts without inventing local dedupe machinery.

There is one nuance worth noticing. If the same node ID is later added with a different label, the existing node is kept and its properties are not merged. That is conservative, but it means label conflicts are silent. For this project, stable IDs are supposed to encode entity type, so silence is acceptable today. For a stricter data-quality pass, Grust could eventually expose a conflict policy.

Store Traits

The central trait is `GraphStore`:

```
#[async_trait]
pub trait GraphStore: Send + Sync {
    async fn put_node(&self, node: &Node) -> Result<NodeId>;
```

```
    async fn put_edge(&self, edge: &Edge) -> Result<Option<EdgeId>>;
    async fn put_graph(&self, graph: &Graph) -> Result<LoadReport>;
    async fn get_node(&self, id: &NodeId) -> Result<Option<Node>>;
    async fn get_edges(&self, query: EdgeQuery) -> Result<Vec<Edge>>;
    async fn traverse(&self, traversal: Traversal) -> Result<Vec<Node>>;
}
```

This trait says Grust is not only a loader abstraction. It wants to be a portable graph access layer: write nodes, write edges, load graphs, fetch nodes, query edges, and traverse.

`GraphAdminStore` adds lifecycle operations:

```
pub trait GraphAdminStore: GraphStore {
    async fn bootstrap(&self) -> Result<()> {
        Ok(())
    }

    async fn clear(&self) -> Result<()>;
}
```

That is exactly the lifecycle this project needs. `bootstrap` creates tables, schemas, graph metadata, or namespaces. `clear` supports `--replace`. `put_graph` does the data write.

The main tradeoff is that `GraphStore` currently asks every backend to expose read and traversal methods, even when a backend only implements writes. Several stores return `GrustError::Unsupported` for reads. That is honest and typed, but it also means there may eventually be room for smaller traits such as `GraphWriteStore`, `GraphReadStore`, and `GraphTraversalStore`.

Error Model

Grust errors are intentionally compact:

```
pub enum GrustError {
    Backend(String),
    Schema(String),
    Unsupported(String),
    Serialization(String),
}
```

For an adapter library, this is a reasonable first layer. It separates backend failures from schema validation, unsupported capabilities, and serialization problems. It also avoids leaking every dependency's native error type into application code.

The cost is that errors are mostly strings after they cross the adapter boundary. That is fine for CLI tools like this one. A larger service might want more structured backend error variants over time.

Backend Adapter Maturity

The backends are not all at the same maturity level, and the book should be honest about that.

FalkorDB and Helix are primarily write adapters today. They batch graph writes and support admin clearing, but `get_node`, `get_edges`, and `traverse` return `Unsupported`.

SurrealDB is also write/admin oriented in the current Grust code. It has both HTTP and SDK stores, shared SurrealQL generation, namespace/database bootstrap, and table clearing. Portable reads and traversal are not implemented yet.

LanceDB is more complete. It stores nodes and edges as Arrow-backed tables, serializes `Props` as JSON, supports merge-insert upserts, implements `get_node`, `get_edges`, and performs traversal by querying edge rows and following node IDs. This makes it especially useful as a local durable backend for examples and tests.

pgGraph is also more complete. It stores nodes and edges in PostgreSQL tables, uses `jsonb` for properties, validates schema and table identifiers, supports upserts, can call `graph.build(...)`, and compiles Grust traversals into SQL joins. Its two build modes, `csr_readonly` and `mutable_overlay`, expose real pgGraph behavior without leaking pgGraph details into this project's importer.

Sail writes through Spark Connect into Delta-style tables named `grust_nodes` and `grust_edges`. It also implements reads and traversal with Spark SQL and Arrow batch parsing. One current limitation is visible in edge writes:

```
sql_str("") // src_label unknown at edge time
```

Edges know endpoint IDs, but not endpoint labels. That is enough for many traversals, but it means the Sail edge table does not fully preserve endpoint labels unless the adapter later looks them up or writes graph batches with node context.

The memory store is a simple in-process implementation backed by `RwLock` and `BTreeMap`. It is useful as a correctness oracle and test fixture. CocoIndex is not a store; it is an export adapter that converts Grust graphs into CocoIndex-style node and relationship state and validates that edge endpoints exist.

How Grust Changes This Codebase

Before Grust, this project had to own too much:

- a local graph model
- backend-specific query builders
- backend-specific escaping rules
- backend-specific batching
- backend-specific clear/bootstrap behavior

After Grust, this project owns the parts that are truly By the Bay specific:

- scrape and enrichment formats
- conversion from export records into `Graph`
- CLI flags and defaults
- label and relationship vocabulary
- backend selection

That is the right separation. Grust is where graph portability lives. This project is where By the Bay domain knowledge lives.

The design is not finished, but it is pointed in a good direction. The most useful next improvements would be:

- split write/read/traversal capability traits if unsupported reads become too common
- expose backend capability metadata so CLIs can explain what a selected store can do
- make label-conflict handling in `GraphBuilder` explicit
- add more integration tests that compare traversal behavior across Memory, LanceDB, pgGraph, and Sail

- keep moving backend-specific escaping and validation into backend crates

The important thing is that the abstraction now protects a real boundary. Grust does not hide the fact that backends differ; it gives those differences a stable place to live.

38. Functional Programming in the Codebase

The project uses a practical Rust version of functional programming: iterators, maps, filters, folds-by-collection, and expression-oriented transforms.

Consider `source_nodes`:

```
source_urls
    .iter()
    .filter(|url| seen.insert((*url).clone()))
    .map(|url| Node::new(/* ... */))
    .collect()
```

This pipeline says: borrow each URL, keep only the first occurrence, transform it into a `Node`, and collect the result.

Consider the JSON property conversion:

```
values
    .into_iter()
    .filter_map(|value| match value {
        serde_json::Value::String(value) if !value.is_empty() => Some(value),
        _ => None,
    })
    .collect()
```

`filter_map` combines validation and transformation. Invalid array entries disappear. Valid strings move into the output vector.

Rust's iterator style is not merely cosmetic. It controls ownership. `iter()` borrows. `into_iter()` consumes. `cloned()` makes owned copies at visible points.

39. Option as a Data-Cleaning Tool

The project uses `Option<T>` for missing data, but also as a small functional pipeline:

```
fn insert_optional(props: &mut Props, key: &str, value: Option<&str>) {
    if let Some(value) = value.filter(|value| !value.is_empty()) {
        props.insert(key.to_string(), value.into());
    }
}
```

This helper reuses one rule everywhere: only insert optional string properties when they exist and are not empty. The function takes `Option<&str>`, not `Option<String>`, because callers often hold borrowed data:

```
insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
```

The scraper uses `Option` similarly:

```
title.map(clean_text).and_then(nonempty)
```

`map` transforms the value if it exists. `and_then` chains a transformation that may itself reject the value.

40. Result and Error Context

The project uses `anyhow::Result` for application-level errors. This is appropriate because the binaries need clear failure messages more than they need a stable typed error API.

The code often adds context at I/O boundaries:

```
fs::read_to_string(input)
    .with_context(|| format!("failed to read {}", input.display()))?;
```

The `?` operator returns early if the operation failed. `with_context` preserves the original error and adds local meaning. The loader also adapts Grust errors into `anyhow`:

```
fn grust_error(err: GrustError) -> anyhow::Error {
    anyhow::Error::new(err)
}
```

That keeps the binary's error surface consistent while still letting Grust own backend-specific error types.

41. Ownership: Moves, Clones, and Reuse

The project clones in places where duplicated ownership is simpler than lifetime coupling:

```
redis_url: args.redis_url.clone()
```

`Args` and `GraphLoadConfig` both own their strings. That makes the config easy to pass around without borrowing from the CLI parser's local value.

The project moves values when input ownership is no longer needed:

```
nodes_by_id.into_values().collect()
```

After deduplicating nodes in a map, the map itself is no longer useful. `into_values` consumes it and moves each `Node` into the output vector.

The project borrows when it only needs to inspect:

```
for node in &graph.nodes {
    // read node and write backend query
}
```

42. Small Helper Functions as Policy Reuse

The most successful reuse in the project is small and direct.

`props` removes repeated map boilerplate. `insert_optional` centralizes optional-property filtering. `graph_labels` and `graph_relationships` centralize the known By the Bay label and relationship vocabulary passed to stores that need schema hints. `surreal_config` keeps SurrealDB HTTP and SDK configuration in one place.

`load_with_admin_store` is the larger version of the same idea. It centralizes the policy for a graph load: bootstrap, optionally clear, then put the graph. This is not just code reuse. It is policy reuse.

43. Async Rust in a Mostly Synchronous Loader

The scraper is naturally async because it fetches remote resources. It uses Tokio and async `reqwest`:

```
async fn fetch(client: &Client, url: &str) -> Result<String>
```

The loader binary is mostly synchronous because file reading and command-line parsing are simple blocking operations. The backend stores are async, so the code bridges the two worlds by creating a runtime:

```
let runtime = tokio::runtime::Runtime::new()?;
runtime.block_on(load_graph_async(config, graph))?;
```

This keeps the public loader function synchronous. If the project later needs high-concurrency loading, it could expose an async loader as well. Today, the synchronous wrapper is friendlier for a CLI.

44. Generics in the Project

The codebase uses generics where they pay their rent.

The `props` helper uses const generics:

```
fn props<const N: usize>(items: [(&str, Value); N]) -> Props
```

This lets callers pass arrays of any fixed length without heap-allocating a temporary vector.

The scraper uses a type parameter for JSON writing:

```
fn write_source_json<T: Serialize>(raw_dir: &Path, file_name: &str, value: &T) -> Result<()>
```

The LLM extractor uses a higher-ranked trait bound:

```
async fn extract_json<T: for<'de> Deserialize<'de>>(...)
```

The practical meaning is: “give me any type that serde can deserialize from this response text.”

The Grust loading helper is generic over store type:

```
async fn load_with_admin_store<S>(store: &S, replace: bool, graph: &Graph) -> Result<LoadReport>
where
    S: GraphAdminStore,
```

The practical meaning is: “give me any store that can bootstrap, clear, and write a graph.”

45. Pattern Matching and `let else`

Rust pattern matching appears everywhere in the project. The most visible examples are `match` expressions over enums and JSON values.

The scraper also uses `let else` for early exits inside parsing logic:

```
let Some(speaker_text) = speakers.get(&id).map(|s| clean_text(s)) else {
    continue;
};
```

That reads as: if this optional value exists, bind it; otherwise continue the loop. It is clearer than a nested `if let` when the rest of the loop requires the value.

46. Collections and Determinism

The project often uses `BTreeMap` and `BTreeSet` instead of hash-based collections. That decision gives deterministic output order and combines deduplication with sorting.

For talk-record import, this key deduplicates edges by source, target, and relationship:

```
BTreeMap<(String, String, String), Edge>
```

Determinism matters for generated files, test expectations, and database query debugging. A graph loader that emits stable output is easier to reason about.

47. Backend-Specific Semantics

FalkorDB, HelixDB, LanceDB, pgGraph, Sail, and SurrealDB do not share one query language or one storage model. Some are graph databases, some are table stores, and Sail routes through Spark Connect.

The architecture shares the portable graph and isolates backend semantics behind Grust store adapters. The By the Bay code supplies the data and user-facing configuration; Grust owns protocol translation.

48. Property Model Differences Across Backends

The property model is where backend differences become visible. Grust's `Value` enum can represent nulls, booleans, integers, floats, strings, string arrays, and fallback JSON values:

```
Value::StringArray(vec!["rust".to_string(), "graphs".to_string()])
```

That is why the export includes multiple tag representations: `tags` as `StringArray`, `tags_csv` as a string, and `tags_json` as a string. Backends with array support can use `tags`. Backends with string-only property support still receive useful tag data.

49. Testing as Executable Design Notes

The tests document decisions that are easy to break accidentally.

The import tests assert that export records and talk-record files both become the same neutral Grust `Graph` shape. They also assert important property details, such as `tags` staying a `Value::StringArray` and talk-record nodes getting both `id` and `nid` properties.

These tests are design locks. They protect the small transformations that would be painful to debug only after a database write failed.

50. Key Rust Features Referenced by the Codebase

This project touches a wide set of practical Rust features:

- `struct` for domain records and configuration
- `enum` for closed choices and portable value types
- `trait` for Grust store behavior
- `impl` blocks for constructors, conversions, and backend implementations
- `derive` macros for `Debug`, `Clone`, `Default`, `Deserialize`, `Serialize`, and `ValueEnum`
- attributes such as `#[arg(...)]`, `#[serde(...)]`, `#[tokio::main]`, and `#[cfg(test)]`
- owned types such as `String`, `PathBuf`, `Vec<T>`, `BTreeMap<K, V>`, and `BTreeSet<T>`
- borrowed types such as `&str`, `&PathBuf`, and `&Graph`
- `Option<T>` for missing data
- `Result<T>` and `?` for fallible code
- iterators: `iter`, `into_iter`, `map`, `filter`, `filter_map`, `flat_map`, `collect`
- closures for inline transformations
- generics and trait bounds
- const generics
- async functions and `.await`
- pattern matching, tuple keys, and `let else`
- modules and crate-level reuse
- tests with `#[test]`

The valuable point is not that the project uses many features. The valuable point is that each feature has a job. Rust is strongest when types, ownership, and module boundaries describe the architecture directly.

51. Edition 3 Summary

The first edition used the loader to explain the borrow checker. Edition 2 showed the wider design. Edition 3 shows the current architecture:

- scrape and extraction code stays at the edge
- exported data becomes a stable contract
- loader input normalizes into Grust `Graph`
- backend stores implement shared Grust traits
- HTTP, SDK, table, PostgreSQL, and Spark-backed paths share one load lifecycle
- functional iterator style keeps transformations compact
- small helpers reuse policy without over-abstracting the system

The result is not a giant framework. It is a practical Rust application whose architecture is visible in its types.